

PROCESSO SEARCHBAR EVENTI E FORUM

Documento di supporto al merge delle modifiche fatte su backend e frontend.

OBIETTIVO GENERALE

Abbiamo introdotto la ricerca in due aree dell'applicazione:

- pagina Eventi;
- pagina Forum.

La ricerca Eventi filtra gli eventi futuri in base a titolo, descrizione o luogo.

La ricerca Forum è più articolata: cerca sia nei titoli delle discussioni sia nel contenuto dei post/commenti, mostra uno snippet del contenuto trovato ed evidenzia visivamente la parola cercata.

BACKEND - EVENTI

EventController.java

È stata modificata la route che restituisce gli eventi.

Prima la route GET /api/events caricava semplicemente la lista degli eventi.

Ora accetta anche un parametro opzionale:

```
-----  
@RequestParam(required = false) String search  
-----
```

Questo significa che la stessa API può essere chiamata in due modi:

```
-----  
GET /api/events  
GET /api/events?search=rock  
-----
```

Se search non viene passato, il comportamento rimane quello originale.

Se search viene passato, il controller lo inoltra al service.

EventService.java

Nel service è stata aggiunta la logica decisionale.

Il metodo che carica gli eventi ora controlla il valore di search:

- se search è null, vuoto o composto solo da spazi, vengono caricati normalmente tutti gli eventi futuri;
- se search contiene testo, viene chiamato il repository con la query di ricerca.

La logica segue questo schema mentale:

1. Il controller riceve il parametro dalla richiesta HTTP.
2. Il service decide quale comportamento applicare.
3. Il repository esegue la query sul database.

Questo mantiene il controller leggero e sposta la logica applicativa nel service.

EventRepository.java

Nel repository e stata aggiunta una query custom per cercare negli eventi futuri.

La ricerca controlla:

- titolo;
- descrizione;
- luogo.

La query mantiene il filtro sugli eventi futuri, quindi non restituisce eventi passati.

E stato mantenuto anche:

```
-----  
@EntityGraph(attributePaths = {"createdBy"})  
-----
```

Questo serve a caricare subito anche l'utente che ha creato l'evento, evitando problemi di lazy loading o query aggiuntive non necessarie.

FRONTEND - EVENTI

event.service.ts

Il metodo getAll() e stato modificato per accettare un parametro opzionale search.

Quando search contiene testo, il service lo manda al backend come query parameter.

Schema:

```
-----  
getAll(search?: string)  
-----
```

Se la ricerca e vuota, viene chiamata la lista normale.

Se la ricerca contiene testo, viene chiamato:

```
-----  
/api/events?search=...  
-----
```

events-list.component.ts

Nel componente della lista eventi e stato aggiunto uno stato per la searchbar.

Sono stati introdotti:

- searchTerm, cioe il testo scritto dall'utente;
- un metodo per reagire al cambio testo;
- il reload degli eventi passando la ricerca al service.

Il flusso e:

1. L'utente scrive nella searchbar.
2. Il componente aggiorna searchTerm.
3. Il componente richiama il service.
4. Il service chiama il backend.
5. La pagina mostra solo gli eventi filtrati.

events-list.component.html

Nel template e stato aggiunto l'input della searchbar nella pagina Eventi.

L'input è collegato al metodo del componente che aggiorna la ricerca.

events-list.component.css

Sono stati aggiunti gli stili della searchbar.

BACKEND - FORUM

La ricerca Forum è stata prima implementata in modo semplice e poi migliorata.

L'obiettivo finale non era solo sapere quale thread contiene una parola, ma mostrare anche dove è stata trovata.

Per questo è stato creato un DTO dedicato.

ForumSearchResultResponse.java

È stato aggiunto il DTO:

```
-----  
ForumSearchResultResponse  
-----
```

Questo oggetto rappresenta un singolo risultato di ricerca.

Campi principali:

```
-----  
threadId  
title  
categoryName  
authorName  
createdAt  
postCount  
matchedPostId  
snippet  
matchType  
-----
```

Significato dei campi:

- threadId: id della discussione da aprire;
- title: titolo della discussione;
- categoryName: categoria del thread;
- authorName: autore del thread;
- createdAt: data di creazione;
- postCount: numero di risposte/post nel thread;
- matchedPostId: id del post dove è stata trovata la parola, se il match è dentro un post;
- snippet: testo da mostrare come anteprima del risultato;
- matchType: indica se il match è nel TITLE o in un POST.

Questo DTO evita di mandare al frontend tutta l'entità ForumThread o Post, e restituisce solo i dati necessari alla UI.

ForumController.java

Il controller espone la ricerca forum tramite una route simile a:

```
-----  
GET /api/forum/search?search=robot  
-----
```

La risposta non e piu una semplice lista di thread summary, ma una lista di ForumSearchResultResponse.

Questo permette al frontend di distinguere:

- risultati trovati nel titolo;
- risultati trovati nel contenuto dei post.

ForumService.java

Nel service e stata aggiunta la logica centrale della ricerca.

Il metodo di ricerca:

1. prende la stringa cercata;
2. la pulisce con trim();
3. se e vuota, ritorna List.of();
4. cerca i thread con titolo corrispondente;
5. cerca i post con contenuto corrispondente;
6. converte entrambi i tipi di risultato in ForumSearchResultResponse;
7. unisce i risultati e li restituisce.

E stato usato:

```
-----  
@Transactional(readOnly = true)  
-----
```

Significa che il metodo apre una transazione di sola lettura.

Serve per dire a Spring/Hibernate che quel metodo legge dati ma non li modifica.

Vantaggi:

- rende piu chiara l'intenzione del metodo;
- puo aiutare Hibernate a ottimizzare;
- evita modifiche accidentali gestite come operazioni di scrittura.

Nel service e stato anche aggiunto un helper mapper, cioe un metodo privato che prende un thread/post e costruisce il DTO ForumSearchResultResponse.

Questo mantiene pulito il metodo principale e centralizza la costruzione della risposta.

ForumThreadRepository.java

Nel repository dei thread e stata aggiunta una query custom.

La query cerca nei titoli dei thread e nei post collegati.

E stata scelta una @Query invece di un metodo derivato tipo findBy... perche la ricerca attraversa piu entita e diventerebbe troppo lunga e poco leggibile.

E stato usato DISTINCT per evitare duplicati quando piu post dello stesso thread contengono la parola cercata.

E stato mantenuto @EntityGraph per caricare autore e categoria del thread.

PostRepository.java

Nel repository dei post e stata aggiunta una query custom che cerca dentro il contenuto dei post.

La query ritorna i post ordinati dal piu recente al meno recente.

Anche qui e stato usato @EntityGraph per caricare:

- autore del post;
- thread collegato;
- autore del thread;
- categoria del thread.

Questo e importante perche il service deve costruire una risposta completa senza inciampare in problemi di lazy loading.

FRONTEND - FORUM

forum.model.ts

E stata aggiunta l'interfaccia:

```
-----  
ForumSearchResult  
-----
```

Serve a rappresentare lato TypeScript la risposta del backend.

Contiene gli stessi campi del DTO backend:

```
-----  
threadId  
title  
categoryName  
authorName  
createdAt  
postCount  
matchedPostId  
snippet  
matchType  
-----
```

matchType puo valere:

```
-----  
'TITLE' | 'POST'  
-----
```

Questo permette al frontend di sapere se il risultato arriva dal titolo della discussione o dal contenuto di un post.

forum.service.ts

E stato aggiunto/modificato il metodo:

```
-----  
searchThreads(search: string)  
-----
```

Questo metodo chiama l'endpoint backend:

```
-----  
/api/forum/search?search=...  
-----
```

e ritorna un array di ForumSearchResult.

threads-list.component.ts

Nel componente della pagina Forum sono stati aggiunti nuovi stati:

```
-----  
searchTerm  
searchResults  
-----
```

searchTerm contiene il testo scritto nella barra di ricerca.

searchResults contiene i risultati restituiti dal backend quando si cerca qualcosa.

La logica di caricamento ora funziona così:

- se searchTerm è vuoto, viene mostrata la lista normale dei thread;
- se searchTerm contiene testo, viene chiamata la ricerca backend e la lista normale viene svuotata.

Questo evita di mostrare contemporaneamente risultati filtrati e lista completa.

Sono stati aggiunti anche helper per l'evidenziazione:

```
-----  
getHighlightedParts(...)  
isHighlightedPart(...)  
-----
```

getHighlightedParts divide il testo in pezzi, separando la parola cercata dal resto.

isHighlightedPart controlla se un pezzo corrisponde esattamente alla ricerca.

Nel template, se un pezzo corrisponde, riceve la classe CSS di highlight.

threads-list.component.html

Il template è stato aggiornato per mostrare due stati diversi.

Quando la searchbar è vuota:

- si vedono le categorie;
- si vede la lista normale delle discussioni.

Quando la searchbar contiene testo:

- si vedono i risultati di ricerca;
- ogni risultato è una card cliccabile;
- ogni card porta al thread corretto;
- viene mostrato lo snippet;
- la parola cercata viene evidenziata.

Ogni risultato mostra:

- categoria;
- se il risultato è nel titolo o in un post;
- titolo;
- snippet;
- autore;
- numero risposte;
- data.

threads-list.component.css

Sono stati aggiunti gli stili per:

- searchbar forum;
- card dei risultati;
- badge categoria;
- snippet;
- highlight della parola cercata.

E stata aggiunta anche una correzione importante su .search-results.

Nel CSS globale esisteva già una classe .search-results pensata per un altro componente, probabilmente con:

```
-----  
position: absolute;  
-----
```

Questo faceva apparire i risultati in fondo alla pagina, come se fossero in un footer.

Per risolvere, nel CSS del componente forum è stato aggiunto un override:

```
-----  
.search-results {  
  display: grid;  
  gap: 1rem;  
  position: static;  
  width: auto;  
  max-height: none;  
  overflow: visible;  
  padding: 0;  
  margin: 0;  
  border: 0;  
  box-shadow: none;  
  background: transparent;  
}  
-----
```

Questa modifica forza i risultati a stare nel flusso normale della pagina, subito sotto la searchbar.

BUG E PROBLEMI RISOLTI

Swagger crea davvero dati

Quando si usa Try it out su Swagger, Swagger non simula la richiesta: la esegue davvero contro il backend.

Per questo una chiamata POST /api/events ha creato davvero un evento visibile sul sito.

404 da browser ma 200 da Swagger

Il problema era il context path.

L'app backend non rispondeva semplicemente a:

```
-----  
http://localhost:8080/api/events?search=jamming  
-----
```

ma a un URL con context path:

```
-----  
http://localhost:8080/suonacongigi/api/events?search=jamming  
-----
```

Swagger usava già l'URL corretto, il browser no.

Risultati Forum invisibili

La ricerca Forum funzionava lato backend e i risultati arrivavano in console.

Il problema era solo visivo: i risultati venivano posizionati in fondo alla pagina a causa della classe globale .search-results.

La correzione è stata fatta nel CSS del componente forum con position: static e altri override.

Search forum troppo generica

All'inizio la ricerca restituiva solo la discussione.

Questo non bastava, perché se una parola era dentro uno dei tanti post di una discussione, l'utente non vedeva subito dove si trovava.

La soluzione è stata introdurre ForumSearchResultResponse con:

- matchedPostId;
- snippet;
- matchType.

Ora il frontend mostra il pezzo di testo in cui la parola è stata trovata.

TEST EFFETTUATI

Backend Eventi

Testato da Swagger.

La ricerca eventi restituisce solo eventi futuri che contengono la parola cercata in titolo, descrizione o luogo.

Backend Forum

Testato da Swagger con JWT tramite pulsante Authorize.

L'inserimento della JWT in Swagger equivale a fare richieste autenticate, perché Swagger aggiunge il token nell'header:

```
-----  
Authorization: Bearer <token>  
-----
```

Frontend Forum

Testato dal sito.

La ricerca:

- chiama correttamente il backend;
- riceve risultati;
- mostra le card nella posizione corretta;

- evidenzia la parola cercata;
- mostra snippet dei post.

GIT E BRANCH

Il lavoro frontend è stato portato sul branch:

```
-----  
dev/SearchBar  
-----
```

Situazione incontrata:

- alcune modifiche erano finite su main;
- il branch dev/SearchBar puntava inizialmente allo stato remoto;
- è stato necessario portare il commit di main dentro dev/SearchBar.

È stato fatto merge del lavoro da main dentro dev/SearchBar.

Poi è stato fatto push su:

```
-----  
origin/dev/SearchBar  
-----
```

PULIZIA CARTELLA FRONTEND NESTED

Nel repository frontend compariva una cartella inutile:

```
-----  
suonacongigifrontend  
-----
```

dentro il repository suonacongigifrontend.

Quella cartella era vuota/inutile e Git la vedeva come submodule rotto.

Il comando:

```
-----  
git rm -r suonacongigifrontend  
-----
```

ha dato errore:

```
-----  
fatal: could not lookup name for submodule 'suonacongigifrontend'  
-----
```

Questo significa che Git la stava trattando come un submodule, ma mancava la configurazione corretta in .gitmodules.

La rimozione corretta è:

```
-----  
git update-index --force-remove suonacongigifrontend  
Remove-Item -Recurse -Force .\suonacongigifrontend  
git add -A  
git commit -m "Remove nested frontend submodule"  
git push origin dev/SearchBar  
-----
```

Da comunicare al team:

la cartella nested suonacongigifrontend non contiene feature, non e codice applicativo, non va mergiata come lavoro utile. Era un residuo strutturale/submodule rotto ed e stata rimossa apposta.

PUNTI DI ATTENZIONE PER IL MERGE

Backend

Controllare conflitti in:

- EventController.java;
- EventService.java;
- EventRepository.java;
- ForumController.java;
- ForumService.java;
- ForumThreadRepository.java;
- PostRepository.java;
- ForumSearchResultResponse.java.

La parte Forum dipende dal nuovo DTO ForumSearchResultResponse.

Se durante il merge manca questo DTO, il backend non compila.

Frontend

Controllare conflitti in:

- forum.model.ts;
- forum.service.ts;
- threads-list.component.ts;
- threads-list.component.html;
- threads-list.component.css;
- event.service.ts;
- events-list.component.ts;
- events-list.component.html;
- events-list.component.css.

La ricerca Forum dipende dall'interfaccia ForumSearchResult.

Se manca il model, TypeScript segnala errori su searchResults.

CSS globale

Prestare attenzione alla classe:

```
-----  
.search-results  
-----
```

Esiste una classe globale con lo stesso nome.

Nel componente Forum e stato necessario sovrascriverla, altrimenti i risultati vengono posizionati in fondo alla pagina.

Durante il merge, se questo override viene perso, il bug visivo ritorna.

Submodule/cartella nested

Se su GitHub o in locale ricompare:

suonacongigifrontend/suonacongigifrontend

e probabilmente il vecchio submodule rotto.

Non va considerato parte della feature.

SCHEMA MENTALE RIUTILIZZABILE

Per aggiungere una searchbar in futuro, il pattern seguito e:

1. Controller: aggiungere @RequestParam(required = false) String search.
2. Service: decidere cosa fare se search e vuoto o valorizzato.
3. Repository: scrivere la query vera.
4. DTO/model: creare una risposta dedicata se la UI ha bisogno di dati diversi.
5. Frontend service: aggiungere il metodo che passa search come query param.
6. Component TS: aggiungere stato e metodo di ricerca.
7. Template HTML: aggiungere input e risultati.
8. CSS: sistemare visualizzazione e stati.
9. Test: Swagger per backend, browser per frontend.
10. Merge: controllare DTO, model, query e CSS.